

outer optimization loop

`jax.grad / optax` over $\log \nu$ (one reverse-mode pass per step)

drives the gradient-based methods

Tesseract contract — one typed `apply` + `vector_jacobian_product` (where gradients are needed)

↓ `apply`

↑ VJP (gradient)

burgers_solver

differentiable spectral
Burgers solver (JAX)

VJP = PDE adjoint

PINN

`pinn_jax` $\rightleftharpoons$ `pinn_pytorch`

VJP = native autograd
(`jax.grad` / `torch`)

fmpe_posterior

amortized flow posterior

trained offline, apply-only
→ posterior ($\nu, A_{IC}, \varphi_{IC}$)

Each component is a versioned, framework-agnostic container sharing the `apply`/VJP pattern.

The two PINN backends share one schema, so swapping them is a single `Tesseract.from_image(...)`; the solver and posterior keep their own contracts.